Project Report - ClinicBook: Secure Clinic Booking System

Course: SECU2000 - Application Security
Student: Juan Felipe Montero Cortes
Date: April 2026

1. Introduction

For this project I built a web app called ClinicBook. Patients can search for doctors, book appointments, and upload medical documents. Doctors can log in to view their schedule and patient files. An admin can approve accounts and check an activity log.

I chose this idea because it has several realistic places where security matters - login, file upload, search, and an admin panel. I tried to apply the security concepts from the course to each one from the start rather than adding them at the end.

2. Application Architecture

The app is split into three layers: a front-end (HTML + Bootstrap + JavaScript), a back-end API (Node.js + Express), and a database (SQLite). The front-end never touches the database directly - everything goes through the API. All security checks happen in middleware on the back-end before anything reaches the database.

The main middleware stack: helmet (security headers), express-session (sessions), express-rate-limit (rate limiting), plus my own auth.js, validate.js, and audit.js files.

Routes: /api/login, /api/register, /api/logout, /api/doctors, /api/appointments, /api/uploads, /api/admin/*.

3. Database Design

Five tables. The assignment needed at least four; I added audit_logs for the security logging feature.

Table	Key Columns
users	id, email (UNIQUE), password_hash, role, full_name, is_approved
doctors	id, user_id (FK -> users), specialty, bio, license_number
appointments	id, patient_id (FK -> users), doctor_id (FK -> doctors), scheduled_at, status, notes

documents	id, appointment_id (FK), uploader_id (FK -> users), original_name, stored_name, mime_type, size
audit_logs	id, user_id (FK -> users), action, ip, user_agent, detail, created_at

Foreign keys are enforced with PRAGMA foreign_keys = ON. All queries use prepared statements.

4. Entry Points

Entry Point	Endpoint	Description
Login form	POST /api/login	Email + password login
Registration form	POST /api/register	Patient and doctor sign-up
Search form	GET /api/doctors	Doctor search by name/specialty
File upload	POST /api/uploads	Upload medical documents (PDF/PNG/JPG)
Appointments API	/api/appointments	Book, view, update, cancel
Admin panel	/api/admin/*	User approval and audit log

5. Security Controls

Authentication and Sessions

Passwords are hashed with bcrypt at cost factor 12. Plain-text passwords are never stored. To avoid leaking whether an email is registered through response timing, bcrypt always runs on login - even when the email doesn't exist - using a pre-generated dummy hash.

req.session.regenerate() is called on every successful login to prevent session fixation. The session cookie is httpOnly, sameSite: lax, and secure in production. Sessions expire after 2 hours.

Authorization

Every protected route uses requireAuth to check for a valid session. Role-specific routes use requireRole('admin') or requireRole('doctor', 'admin'). Data-level ownership checks are also applied - for example, a patient can only download their own files (WHERE id = ? AND uploader_id = ?), so guessing another user's document ID returns nothing. Doctor accounts start with is_approved = 0 and are blocked by requireApproved until an admin approves them.

Input Validation

All inputs are validated server-side with express-validator. Checks include: email format, password complexity (8+ characters, uppercase, number), text length limits, integer ranges, ISO 8601 date format, and that appointment dates are in the future. Failed validation returns 400 and stops the request.

SQL Injection Prevention

Every query uses prepared statements. User input is never concatenated into SQL strings. For example, the search route does db.prepare(sql).all(...params) where name is passed as a ? parameter, not injected into the query string.

File Upload Security

Files are checked in three ways: MIME type whitelist, file extension whitelist, and magic bytes verification using the file-type library (reads the first bytes of the actual file to confirm its type). Files are saved with UUID filenames so the original name never becomes a file path. The uploads/ folder is outside public/ and files are only served through an authenticated download route that checks ownership.

Rate Limiting, Headers, and Error Handling

Login is capped at 10 attempts per IP per 5 minutes. Registration is capped at 5 per IP per hour. helmet sets security headers including CSP, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, and HSTS. The global error handler returns only "An unexpected error occurred" - no stack traces or internal details.

Audit Logging

Every sensitive action is written to audit_logs with user ID, IP, and timestamp. Events logged:

REGISTER, LOGIN_SUCCESS, LOGIN_FAIL, LOGIN_BLOCKED, LOGOUT, APPOINTMENT_BOOKED, APPOINTMENT_CANCELLED, APPOINTMENT_STATUS_UPDATED, DOCUMENT_UPLOADED, DOCUMENT_DOWNLOADED, USER_APPROVED, USER_SUSPENDED. The admin panel shows the last 200 entries.

Data and Secrets

The session secret is in a .env file excluded from the repo. A .env.example is committed with placeholder values. The database and uploads folder are also gitignored. The app only collects what it needs - no date of birth, phone number, or payment information. One acknowledged limitation: the database file and uploads folder are not encrypted on disk. Passwords are safe due to bcrypt, but other data would be exposed if someone accessed the server files directly.

6. OWASP Top 10 Mapping

OWASP	Risk	Mitigation
A01 Broken Access Control	Patients accessing other patients' data; users hitting admin routes	requireRole middleware; ownership checks in all queries
A02 Cryptographic Failures	Plain-text passwords; session cookies over HTTP	bcrypt cost 12; httpOnly, secure, sameSite cookies; secret in .env
A03 Injection	SQL injection via search; path traversal via file names	Parameterized queries; express-validator; UUID filenames
A05 Security Misconfiguration	Default headers; guessable session secret; uploads in public folder	helmet; .env secret; uploads/ outside public/
A07 Auth Failures	Brute force; session fixation; timing-based email enumeration	Rate limiting; session.regenerate(); dummy bcrypt on unknown emails
A09 Logging Failures	No way to detect attacks or compromised accounts	audit_logs table; admin log viewer; failed logins recorded

7. STRIDE Threat Model

Threat	Entry Point	Risk	Mitigation
Spoofing	Login	Attacker uses stolen credentials	bcrypt slows cracking; rate limiting; session replaced on login
Spoofing	Admin panel	Regular user calls admin endpoints	requireRole('admin') returns 403 before handler runs
Tampering	Appointment API	Patient tries to confirm their own appointment	Route requires requireRole('doctor', 'admin')
Tampering	File upload	Attacker renames a script to .jpg	MIME + extension + magic bytes checked; files never executed
Repudiation	All routes	User denies their actions	audit_logs records every sensitive action with user ID and IP
Info Disclosure	Login	Different errors reveal which emails are registered	Always returns "Invalid email or password"; bcrypt always runs
Info Disclosure	File download	Patient guesses another patient's document ID	Ownership check before serving; returns 403 if not the owner
Denial of Service	Login / Upload	Flooding login or uploading large files	Rate limiting; 2 MB file size limit enforced by multer
Elevation of Privilege	Registration	Attacker submits role: 'admin' in request	Validator only accepts patient or doctor; admin set in DB only
Elevation of Privilege	Doctor approval	Doctor accesses data before admin review	is_approved = 0 on register; requireApproved blocks all routes

8. Dependencies and Audit

Package	Purpose
bcrypt	Password hashing
express-session	Session management
helmet	Security headers
express-rate-limit	Rate limiting
express-validator	Input validation
multer	File upload parsing
better-sqlite3	Database with prepared statement support
file-type	Magic bytes verification

9. Conclusion

ClinicBook covers all five required entry points and applies real security controls to six OWASP Top 10 categories. The main gap is the lack of disk encryption for the database and uploaded files, which I documented rather than hiding. In a real deployment I would add disk encryption, put the app behind HTTPS, and use a proper file storage service.

Course learning outcomes covered:

- **CLO1** - OWASP and STRIDE sections analyze threats and mitigations per entry point
- **CLO2** - Input validation, prepared statements, magic bytes, size limits, safe error handling
- **CLO3** - bcrypt + session fixation prevention; role middleware + ownership checks; doctor approval workflow
- **CLO4** - Manual testing: SQL injection attempts, disguised file uploads, rate limiter, unauthorized endpoint access
- **CLO5** - npm audit run; package-lock.json committed; each dependency justified in Section 8
- **CLO6** - Passwords hashed; cookies secured; secrets in .env; encryption gap documented
- **CLO7** - Deployment checklist, backup approach, and incident response steps noted
- **CLO8** - Security designed in from the start, not added after